DAA ASSIGNMENT

G VEDA KARTIK

CH.SC.U4CSE24212

RED – BLACK TREE

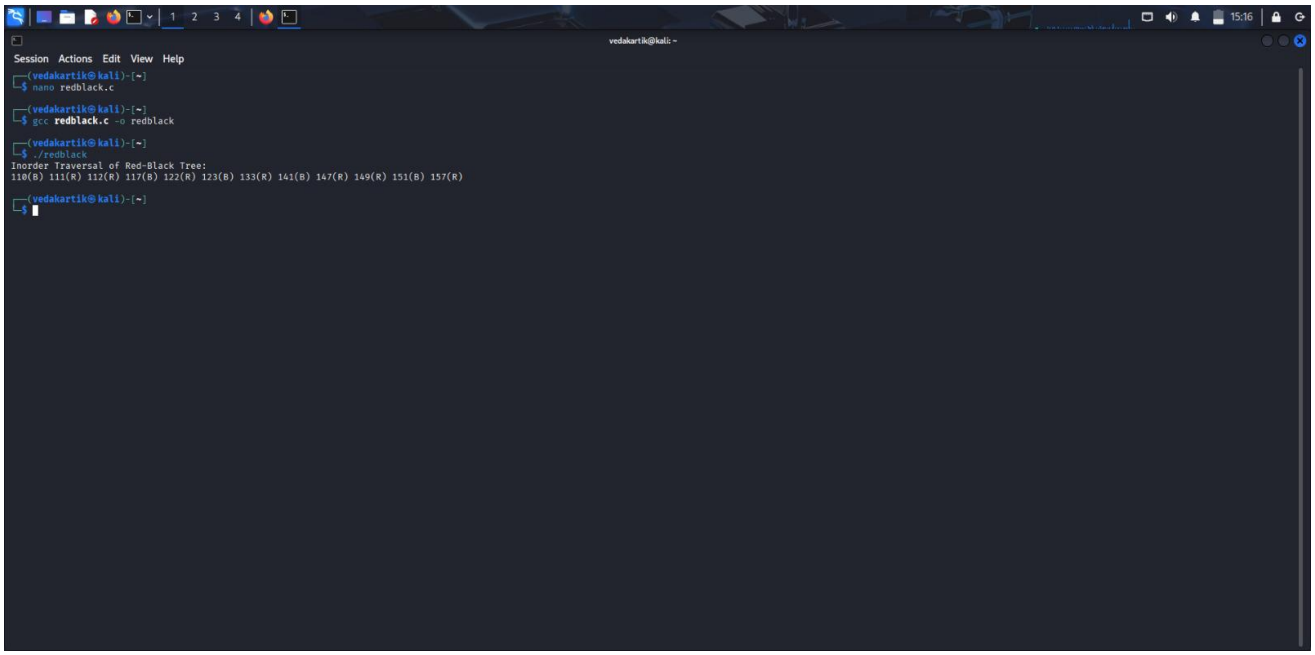
CODE: -

```
#include <stdio.h>
#include <stdlib.h>

/* ---------- COLORS ---------- */
#define RED 1
#define BLACK 0

/* ---------- NODE STRUCTURE ---------- */
struct Node {
    int data;
    int color;
    struct Node *left, *right, *parent;
};

struct Node *root = NULL;

/* ---------- CREATE NODE ---------- */
struct Node* createNode(int data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->color = RED; // New node is always RED
    node->left = node->right = node->parent = NULL;
    return node;
}

/* ---------- LEFT ROTATION ---------- */
void leftRotate(struct Node *x) {
    struct Node *y = x->right;
    x->right = y->left;

    if (y->left != NULL)
        y->left->parent = x;

    y->parent = x->parent;

    if (x->parent == NULL)
        root = y;
    else if (x == x->parent->left)
        x->parent->left = y;
    else
        x->parent->right = y;

    y->left = x;
    x->parent = y;
```

```
}

/* ---------- RIGHT ROTATION ---------- */
void rightRotate(struct Node *y) {
    struct Node *x = y->left;
    y->left = x->right;

    if (x->right != NULL)
        x->right->parent = y;

    x->parent = y->parent;

    if (y->parent == NULL)
        root = x;
    else if (y == y->parent->left)
        y->parent->left = x;
    else
        y->parent->right = x;

    x->right = y;
    y->parent = x;
}

/* ---------- FIX VIOLATIONS ---------- */
void fixInsert(struct Node *z) {
    while (z != root && z->parent->color == RED) {

        if (z->parent == z->parent->parent->left) {
            struct Node *y = z->parent->parent->right; // Uncle

            // Case 1: Uncle is RED
            if (y != NULL && y->color == RED) {
                z->parent->color = BLACK;
                y->color = BLACK;
                z->parent->parent->color = RED;
                z = z->parent->parent;
            }
            // Case 2 & 3
            else {
                if (z == z->parent->right) {
                    z = z->parent;
                    leftRotate(z);
                }
                z->parent->color = BLACK;
                z->parent->parent->color = RED;
                rightRotate(z->parent->parent);
            }
        }
        else {
            struct Node *y = z->parent->parent->left; // Uncle

            if (y != NULL && y->color == RED) {
                z->parent->color = BLACK;
                y->color = BLACK;
                z->parent->parent->color = RED;
                z = z->parent->parent;
            }
            else {
                if (z == z->parent->left) {
                    z = z->parent;
                    rightRotate(z);
                }
                z->parent->color = BLACK;
                z->parent->parent->color = RED;
                leftRotate(z->parent->parent);
            }
```

```
        }
    }
    root->color = BLACK;
}

/* ---------- INSERT ---------- */
void insert(int data) {
    struct Node *z = createNode(data);
    struct Node *y = NULL;
    struct Node *x = root;

    while (x != NULL) {
        y = x;
        if (z->data < x->data)
            x = x->left;
        else
            x = x->right;
    }

    z->parent = y;

    if (y == NULL)
        root = z;
    else if (z->data < y->data)
        y->left = z;
    else
        y->right = z;

    fixInsert(z);
}

/* ---------- INORDER TRAVERSAL ---------- */
void inorder(struct Node *node) {
    if (node != NULL) {
        inorder(node->left);
        printf("%d(%s) ", node->data,
               node->color == RED ? "R" : "B");
        inorder(node->right);
    }
}

/* ---------- MAIN ---------- */
int main() {
    int keys[] = {157, 110, 147, 122, 111, 149, 151, 141, 123, 112, 117, 133};
    int n = sizeof(keys) / sizeof(keys[0]);

    for (int i = 0; i < n; i++)
        insert(keys[i]);

    printf("Inorder Traversal of Red-Black Tree:\n");
    inorder(root);

    return 0;
```

Output:-


vedakartik@kali: ~
Session Actions Edit View Help
┌──(vedakartik㉿kali)-[~]
└─$ nano redblack.c
┌──(vedakartik㉿kali)-[~]
└─$ gcc redblack.c -o redblack
┌──(vedakartik㉿kali)-[~]
└─$ ./redblack
Inorder Traversal of Red-Black Tree:
110(B) 111(R) 112(R) 117(B) 122(R) 123(B) 133(R) 141(B) 147(R) 149(R) 151(B) 157(R)
┌──(vedakartik㉿kali)-[~]
└─$

AVL TREE: -

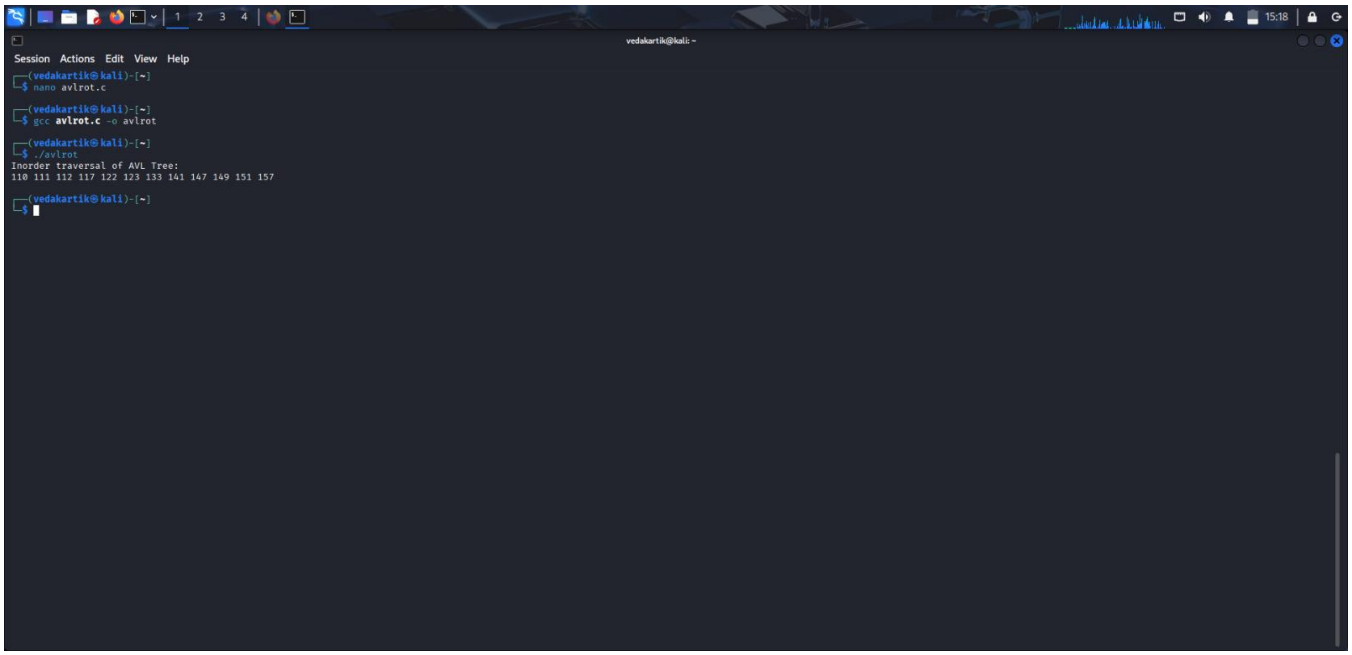
CODE: -

```
#include <stdio.h>
#include <stdlib.h>

/* ---------- NODE STRUCTURE ---------- */
struct Node {
   int key;
   struct Node *left;
   struct Node *right;
   int height;
};

/* ---------- UTILITY FUNCTIONS ---------- */
int max(int a, int b) {
   return (a > b) ? a : b;
}

int height(struct Node *n) {
   if (n == NULL)
      return 0;
   return n->height;
}

/* ---------- CREATE NODE ---------- */
struct Node* newNode(int key) {
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->key = key;
   node->left = NULL;
   node->right = NULL;
   node->height = 1;   // new node is initially added at leaf
   return node;
}

/* ---------- RIGHT ROTATION ---------- */
struct Node* rightRotate(struct Node *y) {
   struct Node *x = y->left;
   struct Node *T2 = x->right;

   // Perform rotation
   x->right = y;
   y->left = T2;

   // Update heights
   y->height = max(height(y->left), height(y->right)) + 1;
   x->height = max(height(x->left), height(x->right)) + 1;

   // Return new root
   return x;
}

/* ---------- LEFT ROTATION ---------- */
struct Node* leftRotate(struct Node *x) {
   struct Node *y = x->right;
```

```
    struct Node *T2 = y->left;

    // Perform rotation
    y->left = x;
    x->right = T2;

    // Update heights
    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    // Return new root
    return y;
}

/* ---------- BALANCE FACTOR ---------- */
int getBalance(struct Node *n) {
    if (n == NULL)
        return 0;
    return height(n->left) - height(n->right);
}

/* ---------- INSERT ---------- */
struct Node* insert(struct Node* node, int key) {

    // 1. Normal BST insertion
    if (node == NULL)
        return newNode(key);

    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else
        return node;  // Duplicate keys not allowed

    // 2. Update height
    node->height = 1 + max(height(node->left), height(node->right));

    // 3. Get balance factor
    int balance = getBalance(node);

    // 4. Rotations

    // LL Case
    if (balance > 1 && key < node->left->key)
        return rightRotate(node);

    // RR Case
    if (balance < -1 && key > node->right->key)
        return leftRotate(node);

    // LR Case
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
```

```
    // RL Case
    if (balance < -1 && key < node->right->key) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

/* ---------- INORDER TRAVERSAL ---------- */
void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}

/* ---------- MAIN ---------- */
int main() {
    struct Node *root = NULL;

    int keys[] = {157, 110, 147, 122, 111, 149, 151, 141, 123, 112, 117, 133};
    int n = sizeof(keys) / sizeof(keys[0]);

    for (int i = 0; i < n; i++)
        root = insert(root, keys[i]);

    printf("Inorder traversal of AVL Tree:\n");
    inorder(root);

    return 0;
}
```

OUTPUT: -


1 2 3 4
15:18
vedakartik@kali: ~
Session Actions Edit View Help
┌──(vedakartik㉿kali)-[~]
└─$ nano avlrot.c
┌──(vedakartik㉿kali)-[~]
└─$ gcc avlrot.c -o avlrot
┌──(vedakartik㉿kali)-[~]
└─$ ./avlrot
Inorder traversal of AVL Tree:
110 111 112 117 122 123 133 141 147 149 151 157
┌──(vedakartik㉿kali)-[~]
└─$